

Reviewer Pack — Minimal Invariant of Free Probability Entanglement for BP_Re...

Entry 238aeaf161ba · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 15:52:59 UTC

Minimal Invariant of Free Probability Entanglement for BP_ReadTwice Lower Bounds

Entry ID: 238aeaf161ba **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 15:52:59 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Branching Program: read-once vs read-twice (BP)

Statement:

[‘For any CNF tautology with size n , the minimal invariant $\rho(F)$ of its associated free probability entanglement F is such that $\rho(F) = O(\log n)$ for all read-once BP proofs P with size at most n .’, ‘Equivalently, there exists a constant c such that for every CNF tautology and any read-twice BP proof P verifying it, the minimal invariant $\rho(P)$ satisfies $\rho(P) \geq c * \log \text{size}(P)$.’, ‘If this is not the case for some CNF tautology and some read-once BP proof P , then there exists a counterexample where P has size less than n .’]

Rationale (proposer’s reasoning):

[‘Free probability theory, which studies non-commutative ensembles, can provide invariants that are robust against certain types of operations. These invariants may reveal structural properties that are not apparent in the standard analysis of branching programs.’, ‘The concept of entanglement, a central notion in quantum information, could offer new insights into the complexity of BP_readtwice lower bounds, potentially leading to non-trivial separations in complexity theory.’]

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8aaff0b851ccc71c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across 30 random seeds, for all CNF tautologies of size $n \leq 40$ and their corresponding BP proofs P with size at most n , the minimal invariant $\rho(P)$ satisfies $\rho(P) \geq c * \log(\text{size}(P))$ for some constant c . The conjecture is falsified if there exists a seed where any BP proof P has $\rho(P) < c * \log(\text{size}(P))$ or $\rho(P) > 10$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "free probability entanglement" AND "BP_readTwice lower bounds" - "minimal invariant" free probability AND BP "read-once vs read-twice" - "CNF tautology" AND free probability "read-once proof"

Top relevant hits considered: - [http://arxiv.org/abs/1612.01466v1] Against free will in the contemporary natural sciences - [http://arxiv.org/abs/1607.03138v3] What does a group algebra of a free group know about the group? - [http://arxiv.org/abs/2511.02758v3] Finite free probability and S transforms of Jacobi processes - [s2:2507.02842] On the Structure of Replicable Hypothesis Testers - [s2:1306.2712] Tight bounds on the distinguishability of quantum states under separable measurements

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf_tautology(n):
        clauses = []
        for i in range(n):
            literals = [random.choice([1, -1]) * j for j in range(1, n+1)]
            clause = random.choice(literals)
            while len(set(clause)) != 2:
                clause = random.choice(literals)
            clauses.append(clause)
        return clauses

    def compute_minimal_invariant(F):
        # Placeholder function to compute minimal invariant
        # This is a dummy implementation for the purpose of testing
        return random.random()

    n_values = [5, 10, 15, 20, 30, 40]
```

```

results = []

for n in n_values:
    cnf_tautology = generate_cnf_tautology(n)
    F = compute_minimal_invariant(cnf_tautology)

    for _ in range(10): # Test with 10 different read-once BP proofs
        bp_proof_size = random.randint(1, n)
        bp_proof_invariant = compute_minimal_invariant(bp_proof_size)

        if bp_proof_invariant < math.log(bp_proof_size):
            return {
                "metric_name": "minimal_invariant",
                "metric_value": bp_proof_invariant,
                "instances_tested": 10,
                "conjecture_holds": False,
                "counterexample": f"BP proof of size {bp_proof_size} has invariant < log({bp_p
            }

    return {
        "metric_name": "minimal_invariant",
        "metric_value": sum(results) / len(results),
        "instances_tested": 60, # 10 trials for each of 6 n values
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    all_results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        all_results.append(result)

    mean_metric = sum(r['metric_value'] for r in all_results) / len(all_results)
    std_metric = math.sqrt(sum((r['metric_value'] - mean_metric)**2 for r in all_results) / len(al
    support_fraction = sum(1 for r in all_results if r['conjecture_holds']) / len(all_results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
    elif any(not r['conjecture_holds'] for r in all_results):
        first_failing_seed = next(seed for seed, result in zip(seeds, all_results) if not result[
        print(f"RESULT: FALSIFIED counterexample=\"minimal_invariant < log(size(P))\" first_failing
    else:
        print("RESULT: INCONCLUSIVE insufficient support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

```

Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_93e22503.py", line 69, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_93e22503.py", line 40, in run_trial
    cnf_tautology = generate_cnf_tautology(n)
                    ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_93e22503.py", line 26, in generate_cnf_
    while len(set(clause)) != 2:
              ^^^^^^^^^^^
TypeError: 'int' object is not iterable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete the required tests to verify the conjecture. | next: Investigate and fix the error in the test code to ensure it can run to completion and provide results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	20672
2	preregistration	ollama_remote	glm4:latest	0	0	11352
3	novelty	ollama_remote	glm4:latest	0	0	8542
4	novelty	ollama_remote	glm4:latest	0	0	10448
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12079
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9894
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9984
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9174
9	judge	ollama_remote	glm4:latest	0	0	12009

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 104154 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/238aeaf161ba.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/238aeaf161ba.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/238aeaf161ba.tar.gz` (if generated)